

School of Engineering

CS570: Big Data
Processing & Analytics
Spring 2026

Week 2: Hadoop Deep Dive

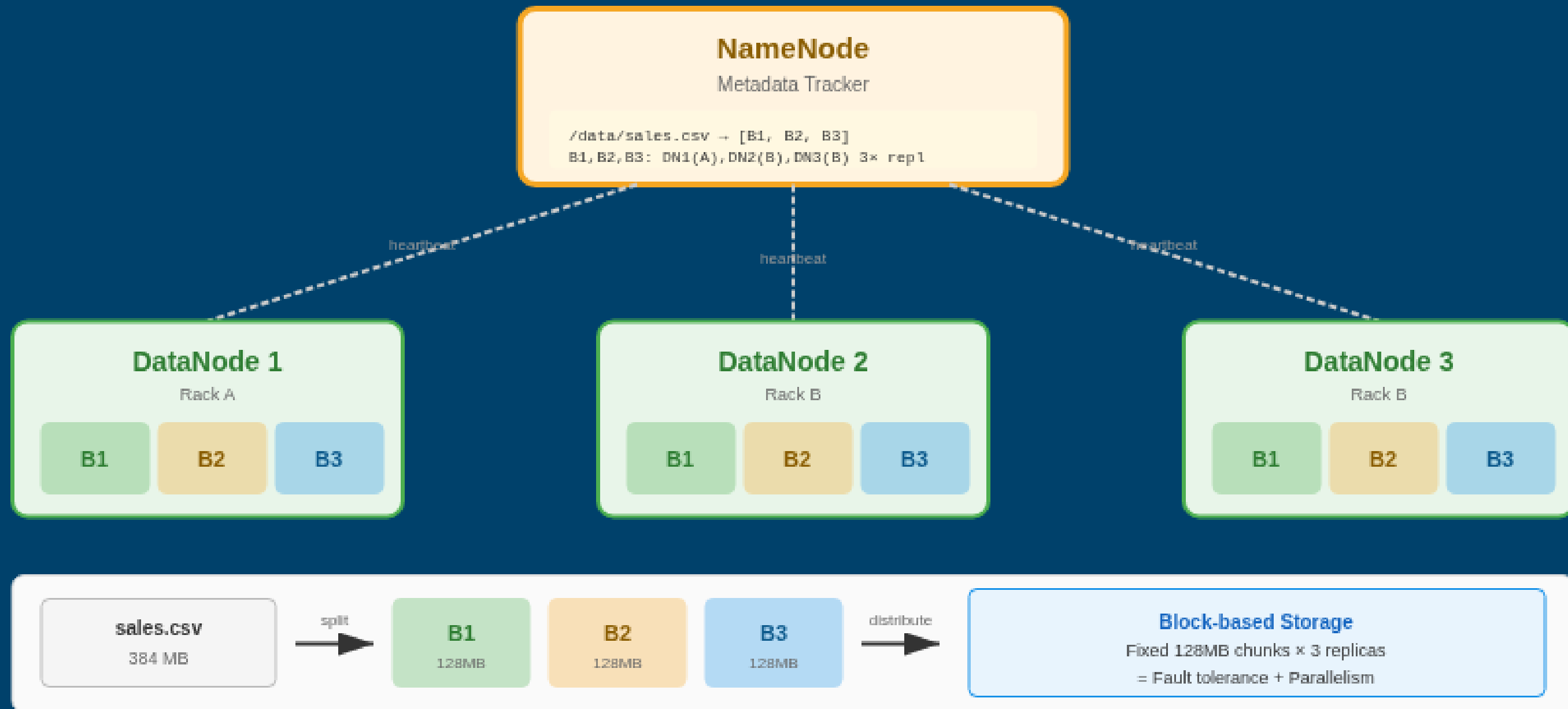


Dr. Ragnar Lesch
Adjunct Professor



HDFS Architecture

Distributed file system components



Simplified: 3 nodes shown. Production racks have 40-50 nodes each.

HDFS Architecture

Distributed file system components

- NameNode: The metadata tracker, it knows which blocks are where, like a librarian's catalog
- DataNodes: Where raw data blocks live, the actual storage shelves holding 128MB chunks
- Data Locality: Computation runs on the same machine where data lives thus avoids network transfer
- Block-based Storage: Files split into fixed-size blocks, distributed across cluster

NameNode: The Metadata Tracker

Central coordinator for HDFS

- Tracks file-to-block mapping: 'data.csv' = blocks 1, 2, 3, 4 on which DataNodes
- Maintains directory tree: File names, permissions, modification times—like a file system
- Receives heartbeats from DataNodes: Every 3 seconds—'I'm alive and have these blocks'
- Single point of failure (prior to High Availability configuration): If NameNode crashes, entire HDFS becomes inaccessible

DataNodes: The Storage Workers

Where data actually lives

- Store actual data blocks: Raw 128MB chunks on local disk—billions of blocks across cluster
- Serve read/write requests: Clients connect directly to DataNodes to transfer data
- Report to NameNode: Send block reports and heartbeats—'I have blocks 42, 157, 891...'
- Execute replication: NameNode says 'copy block 42 to DataNode 5'—DataNode handles transfer

3× Replication Strategy

Every block exists on three different DataNodes

- Default replication factor: 3 copies of every block—configurable but 3 is standard
- Placement strategy: First copy on local node, second on different rack, third on same rack as second
- Write pipeline: Client writes to one DataNode, which pipelines to second, then third—parallel writes
- Trade-off: Uses 3x storage space but provides high availability—lose 2 machines, still have data

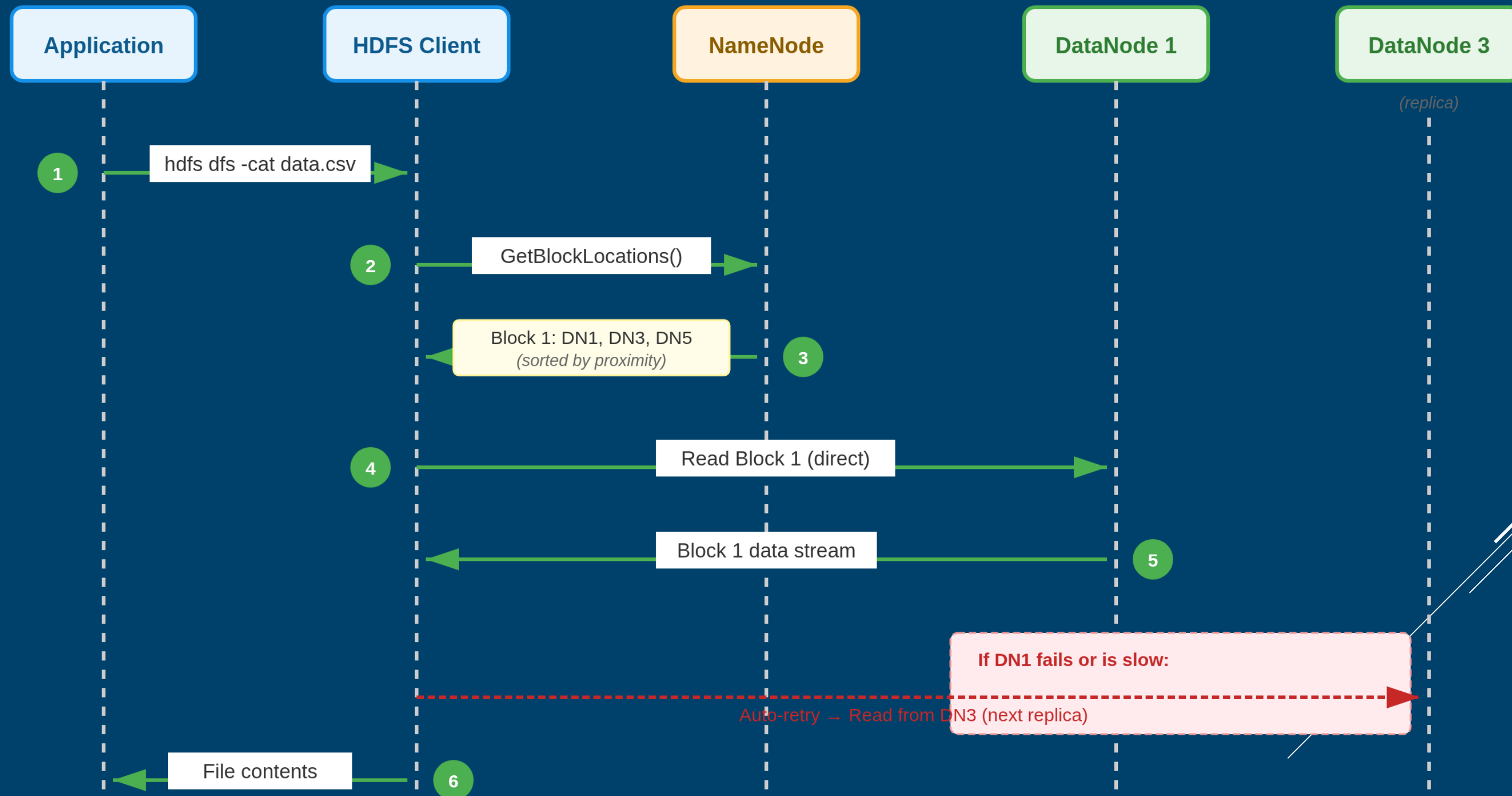
Rack Awareness

Surviving complete rack failures

- Rack = physical grouping: Servers in same rack share network switch and power—correlated failures
- Smart placement: 2 copies on one rack (bandwidth), 1 copy on different rack (fault tolerance)
- Why this matters: Entire rack can fail—power outage, switch failure, accidental disconnect
- Network optimization: Intra-rack bandwidth (1-10 Gbps) faster than inter-rack—minimize cross-rack traffic

How Data Is Read

Client-driven retrieval with automatic failover



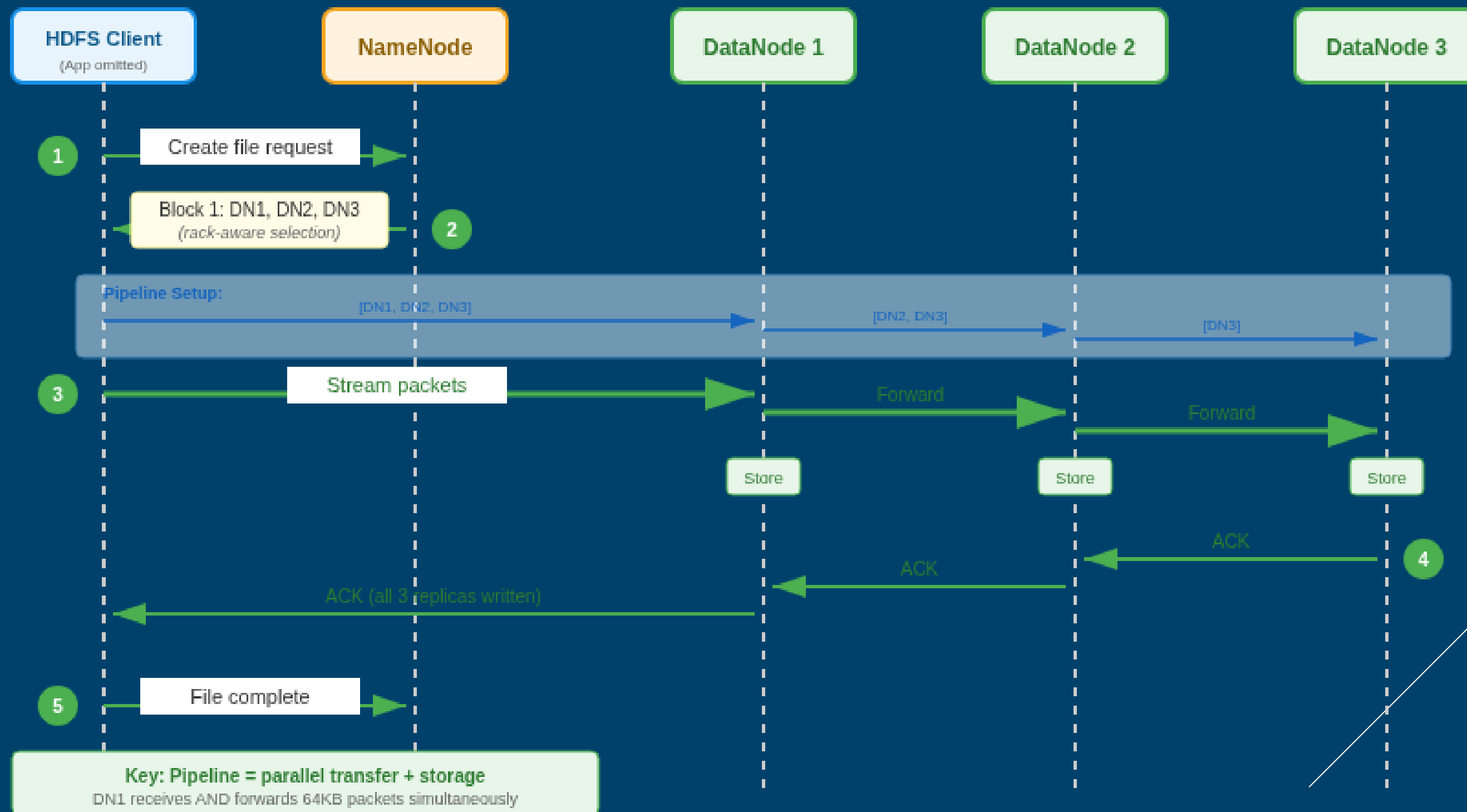
How Data Is Read

Client-driven retrieval with automatic failover

- **Metadata lookup first:** Client asks NameNode for block locations — NameNode returns ordered list of DataNodes per block
- **Proximity-sorted replicas:** NameNode returns DataNodes ordered by network distance — same rack first, then nearby racks
- **Direct DataNode reads:** Client connects directly to DataNodes for data — NameNode never in the data path, no bottleneck
- **Transparent failover:** If DataNode is slow or unavailable, client automatically tries next replica
- **Parallel block reads:** Different blocks can be read from different DataNodes simultaneously (scales with file size)
- **Contrast with writes:** Reads are parallel and direct; writes use sequential pipeline (DN1→DN2→DN3) for consistency

How Data Is Written

Chained writes for 3x replication



How Data Is Written

Chained writes for 3x replication

- **NameNode orchestrates placement:** Client requests to create file, NameNode selects DataNodes using rack-awareness (2 on one rack, 1 on another)
- **Pipeline architecture:** Client streams to DN1, DN1 forwards to DN2, DN2 forwards to DN3; data flows while being stored
- **Parallel transfer + storage:** DN1 receives packets AND forwards simultaneously, doesn't wait until fully received
- **Acknowledgement chain:** ACKs flow back through pipeline (DN3→DN2→DN1→Client); client knows when all replicas are durable
- **Why pipeline, not parallel?** Client has limited upload bandwidth, pipeline uses client's bandwidth once, DataNode-to-DataNode links do the rest
- **Failure handling:** If DN in pipeline fails, client gets new DN list from NameNode and rebuilds pipeline, partial data is cleaned up

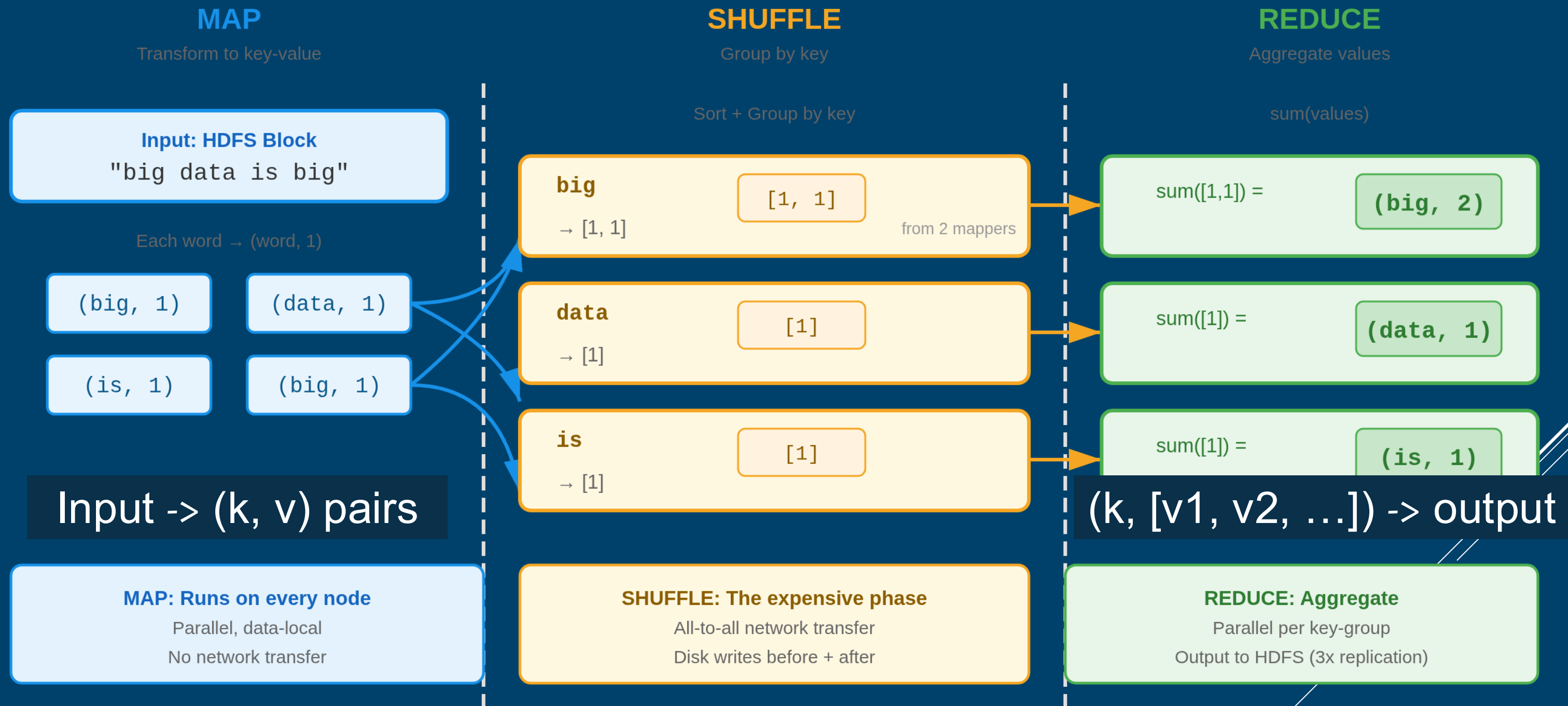
Self-Healing System

Automatic failure detection and recovery

- Heartbeat monitoring: DataNode sends heartbeat every 3 seconds — if NameNode misses 10, declares dead
- Under-replication detection: NameNode scans for blocks with <3 copies—schedules re-replication
- Automatic re-replication: NameNode tells healthy DataNode to copy block to new location
- No human intervention: System continuously monitors and heals itself—critical for large clusters

The MapReduce Model

Three phases for distributed computation



⚠ Disk I/O at every boundary: Map writes → Shuffle reads/writes → Reduce reads → HDFS writes (×3)
This is why iterative algorithms (ML, PageRank) are slow in MapReduce → Spark keeps data in memory

The MapReduce Model

Three phases for distributed computation

Map Phase:

- **Input:** Records from HDFS blocks—each mapper processes one 128MB block
- **User function:** Transform each record into (key, value) pairs
- **Parallel:** 100 mappers process 100 blocks simultaneously—data locality ensures local reads

Shuffle Phase:

- **Purpose:** Group all values with same key across all mappers
- **Partitioning:** $\text{hash}(\text{key}) \% \text{num_reducers}$ determines reducer assignment
- **Why expensive:** Disk writes after map + network transfer + disk writes before reduce

Reduce Phase:

- **Input:** Key + all values for that key—e.g., (big, [1,1,1,1])
- **User function:** Aggregate values—sum, count, max, average, custom logic
- **Parallelism:** Each reducer handles different keys independently

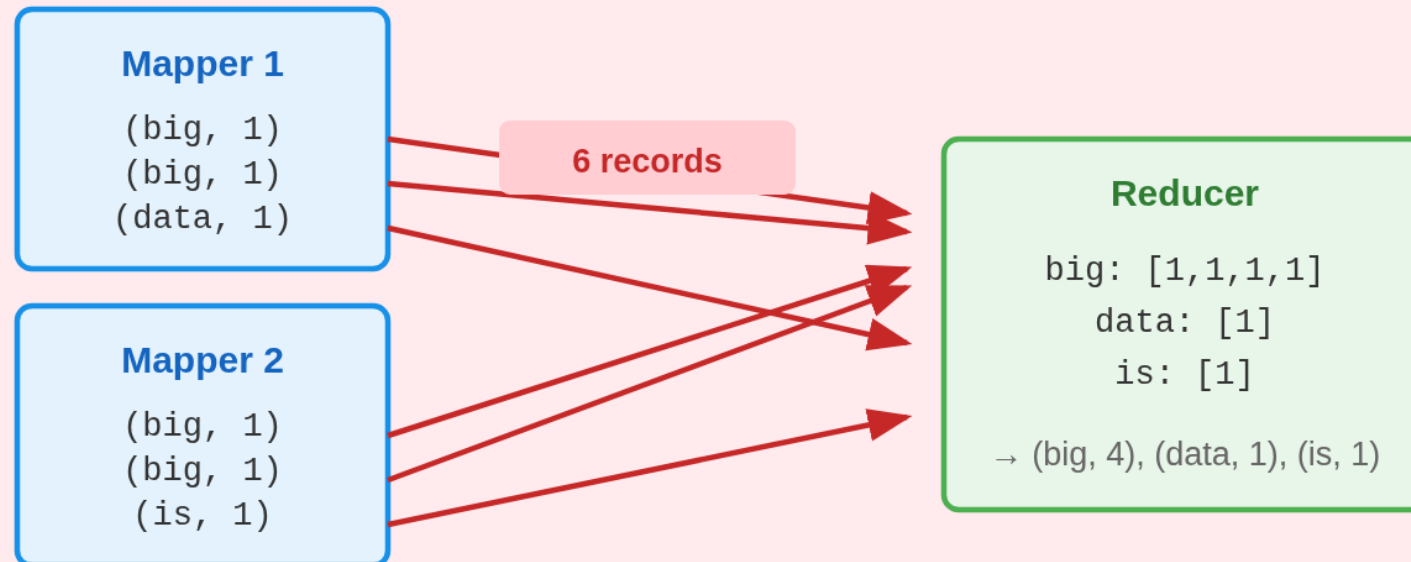
Key insight:

Simple programming model hides distributed systems complexity from developers

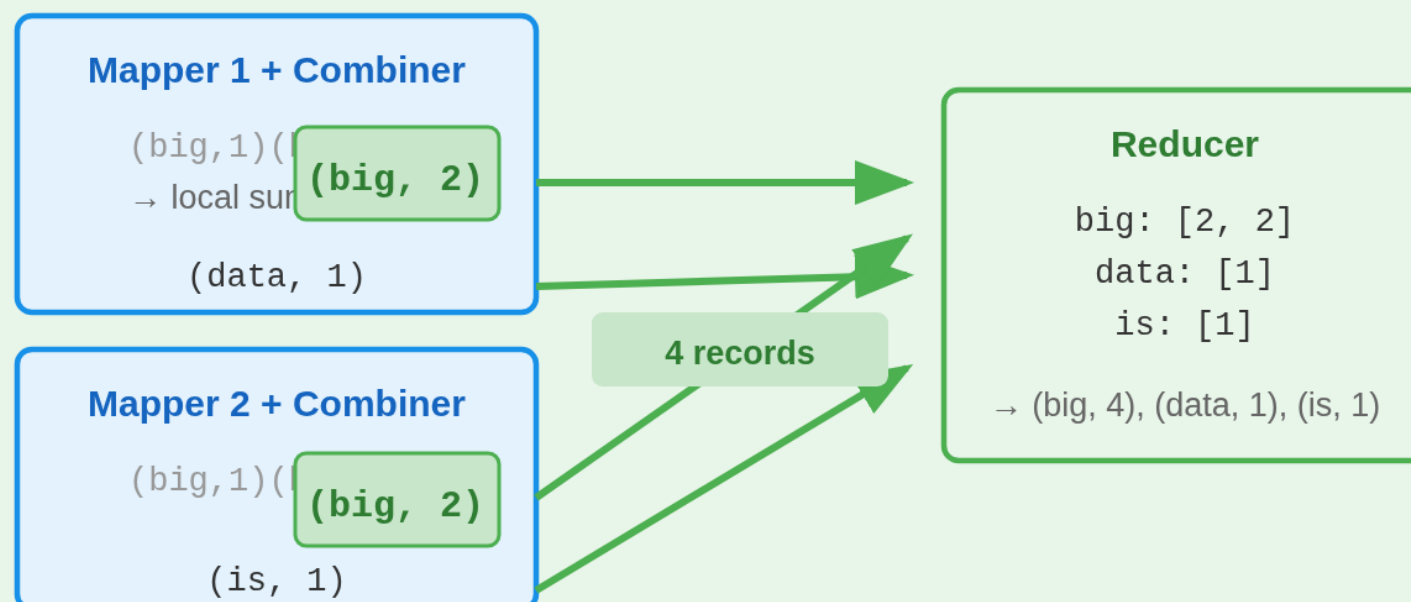
Combiners

Local pre-aggregation to reduce shuffle traffic

✗ Without Combiner



✓ With Combiner



Combiner = Local Reduce

WHEN TO USE

- Reduce function is associative + commutative
- sum, max, min, count → ✓ works
- average → ✗ (but sum+count works)

BENEFITS

- Reduces network transfer (often 10-100x)
- Reduces shuffle disk I/O
- Same result, much faster

HOW IT WORKS

- Runs after map, before shuffle
- Same code as reduce function
- May run 0, 1, or multiple times
- Framework decides when to apply

Real-world: WordCount on Wikipedia → 90% less network traffic

Combiners

Local pre-aggregation to reduce shuffle traffic

- **What it is:** Local pre-aggregation to reduce shuffle traffic
- **When to use:** Only when reduce function is associative and commutative—sum, max, min, count work; average and median don't
- **How it works:** Framework runs combiner 0, 1, or multiple times—you don't control when, so result must be correct regardless
- **Network savings:** Instead of sending (big,1), (big,1), (big,1), (big,1) send just (big,4)—often 10-100x reduction
- **Real-world impact:** WordCount on large text gets 90%+ reduction in network traffic; essential for aggregation workloads
- **Rule of thumb:** If your reduce function is just summing/counting/max/min, always add a combiner, it's free performance.

MapReduce vs Spark

Disk-based vs in-memory processing

- MapReduce: Every operation writes to disk — reliable but slow for iteration
- Spark: Keep intermediate results in RAM — often 10-100x faster for iterative workloads
- Use MapReduce when: Single-pass batch jobs, maximum reliability needed, limited RAM
- Use Spark when: Iterative algorithms, interactive queries, machine learning, sufficient RAM